Part 1

APRENDE PYTHON

✦ Fundamentos ✦



APRENDE PYTHON

✦ Fundamentos ✦

¿Nunca programaste? Empezamos desde cero. Antes de calcular nada, Python necesita que le **demos los datos**. Guardamos tres novelas, cada una con su **título** y su **rating** (su calificación de 0 a 10):

```
novelas = [
  {"titulo": "Rubí", "rating": 8.9},
  {"titulo": "Teresa", "rating": 9.0},
  {"titulo": "Rebelde", "rating": 9.8}
]
```

`[ ]` **Lista:** guarda varias cosas juntas (las 3 novelas).

`{ }` **Diccionario:** los datos de UNA novela.

`"titulo"` **Clave:** el nombre de cada dato.

`8.9` **Valor:** el dato en sí.

💡 `"titulo"` y `"rating"` son nombres que **inventamos nosotros**, no comandos de Python. Pudimos llamarlos `"nombre"` y `"nota"` y funcionaría igual.

Variables · Tipos · Operadores · if / for

Milagros Espinoza Cunya

APRENDE PYTHON
Fundamentos

1 Variables

¿Qué es una variable?

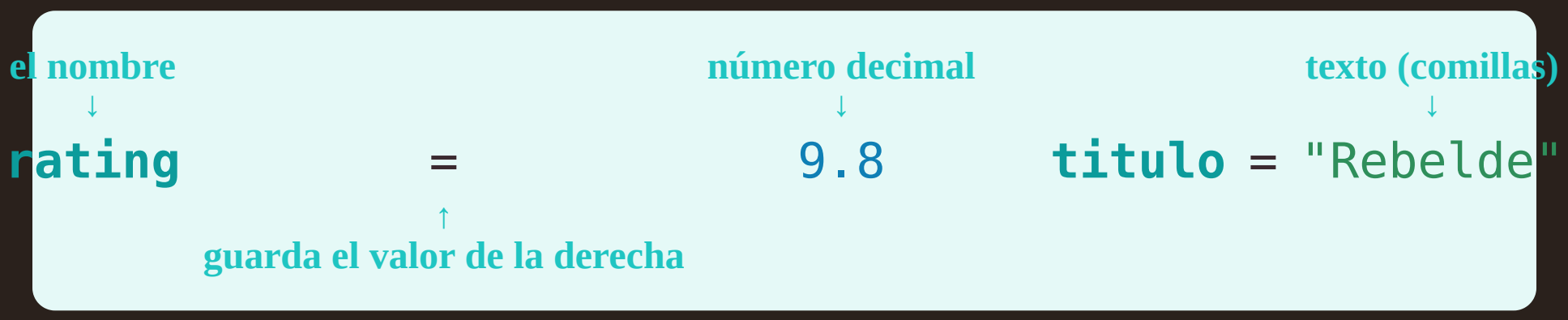
Es como una **caja con etiqueta**: guardas un dato dentro y le pones un nombre para volver a usarlo cuando quieras.

Problema

Queremos guardar el rating y el título de una novela para usarlos después.

La Solución:

Guardamos cada dato en una variable con **=**. Luego usamos **type()** para ver de qué tipo es cada uno.



`type()` es una **función**: le das un dato y te dice de qué tipo es.

Resultado:

```
>>> type(rating)
<class 'float'>
>>> type(titulo)
<class 'str'>
```

DATAZO

El texto SIEMPRE va entre comillas; el número no. Por eso **"9.8"** es texto, y **9.8** es un número con el que sí puedes calcular.

Milagros Espinoza Cunya

APRENDE PYTHON
Fundamentos

2 Tipos de dato

¿Por qué importan?

Cada dato en Python es de un **tipo**. Saber cuál es te dice qué puedes hacer con él (sumar, comparar, unir...) y te evita errores.

`int`	**Número entero.** Sin decimales.	`115`
`float`	**Número decimal.** Con punto.	`9.8`
`str`	**Texto (string).** Va entre comillas.	`"Rebelde"`
`bool`	**Booleano.** Solo verdadero o falso.	`True`

Ejemplo:

```
episodios = 115       # int
rating    = 9.8       # float
titulo    = "Rebelde"  # str
es_exito  = True      # bool
```

DATAZO

No puedes sumar un número con un texto directamente: **9.8 + "hola"** da error. Primero hay que convertir el tipo con funciones como **int()** o **str()**.

Milagros Espinoza Cunya

APRENDE PYTHON
Fundamentos

3 Operadores

¿Qué es un operador?

Es un símbolo que **compara dos valores** y responde solo una cosa: **True** (verdadero) o **False** (falso).

Problema

Queremos saber si "Rebelde" (rating **9.8**) supera el nivel de un éxito, que ponemos en **9.0**.

La Solución:

Usamos el operador **>** (mayor que) para que Python compare y responda sí o no.

```
rating     >     9.0
  ↑        ↑      ↑
el dato que  ¿es mayor  el número
guardamos (9.8)  que…?  a superar
```

> mayor que **< menor que** **== igual a** **! = distinto de**

= guarda un valor · **==** pregunta si son iguales (¡son DOS signos!). El **True/False** que sale es de tipo **bool**.

Resultado:

```
>>> rating > 9.0
True   # 9.8 sí es mayor que 9.0
```

DATAZO

El error más común al empezar: **==** (doble igual) pregunta si dos cosas son iguales; **=** (uno solo) guarda un valor. No son lo mismo.

Milagros Espinoza Cunya

APRENDE PYTHON
Fundamentos

4 Condicionales

¿Qué es un condicional?

Es darle a Python el poder de **decidir**. En español: **SI** pasa algo, haz una cosa; **si no** (else), haz otra.

Problema

Queremos que Python etiquete cada novela como **"Éxito"** o **"Regular"** según su rating.

La Solución:

Usamos **if / else**. Y **print()**, otra función, que sirve para **mostrar** algo en pantalla.

```
if rating > 9.0:
    print("Éxito")
else:
    print("Regular")
```

`if` → SI se cumple… | `:` → abre el bloque | `else` → si NO, haz esto

La **sangría** (los espacios al inicio) le dice a Python qué línea pertenece a cada bloque.

Resultado:

```
Éxito   # entró al if porque 9.8 > 9.0
```

DATAZO

La sangría (esos 4 espacios) NO es decoración en Python: define qué código está "dentro" de cada bloque. Si la olvidas, da error.

APRENDE PYTHON
Fundamentos

5 Bucles (for)

¿Qué es un bucle?
Es **repetir** una acción por cada dato de una lista, sin copiar y pegar el código una y otra vez.

Problema
Tenemos 3 novelas (o 3.000) y queremos mostrar el título de todas, sin escribir **print** una por una.

La Solución:
Un bucle **for** recorre la lista y hace lo mismo con cada novela, automáticamente.

```
for novela in novelas:
    print(novela["titulo"])
```

`for` → para cada… | `novela` → cada elemento | `in novelas` → de la lista

`novela["titulo"]` significa: de esta novela, dame el valor de la clave `"titulo"`.

Resultado:

```
Rubí
Teresa
Rebelde   # recorrió las 3 solito
```

DATAZO
Un **for** te ahorra escribir el mismo código cientos de veces. Es la base de casi todo el análisis de datos con Python.

Milagros Espinoza Cunya

APRENDE PYTHON

✦ Fundamentos ✦

Guárdalo. Estas 5 herramientas son la base de TODO lo que verás en Python:

Situación	Herramienta	¿Qué hace?
Guardar un dato	`nombre =`	Mete un valor en una "caja" con nombre (variable).
Saber qué es	`type()`	Dice si es int, float, str o bool.
Comparar	`> < ==`	Responde True o False a una pregunta.
Decidir	`if / else`	Ejecuta un código u otro según la condición.
Repetir	`for`	Recorre todos los datos sin repetir el código.

¡Guarda este post, dale like, compártelo y sígueme para la Parte 2!

Milagros Espinoza Cunya